

Experiment 2: Software Cost Estimation

Part I: Software Size Measurement | Part II: Software Cost Estimation

Student: 胡浩杰 (Haojie Hu) | Student ID: 2351493 | Course Project: HeartCompass

Software Engineering Management and Economics | June 2026

Primary: NESMA Estimated FPA (ISO/IEC 24570) + WBS Backward Tracking | Verification: IFPUG Detailed + GB/T 36964 + COCOMO II

Part I: Software Size Measurement

1. Experimental Background

HeartCompass is a digital persona platform that constructs and maintains AI-powered virtual representations of real individuals. The system ingests multi-source personal data (chat logs, narratives, social media, artifacts), extracts fine-grained personality traits across 4 dimensions (Personality, Interaction Style, Procedural Info, Memory), and enables real-time AI conversations consistent with the constructed persona. The platform integrates with Lark/Feishu for message delivery, uses LangGraph for conversation orchestration, pgvector for semantic recall, and Docker Compose for deployment. Development spans 11 weeks (W1–W11) by a 4-person team, with a cumulative input of 220 person-hours and a weighted completion rate of 94.8% as of Week 11.

Software size measurement is foundational to cost estimation. This experiment applies the NESMA function point analysis framework (ISO/IEC 24570:2018) to measure HeartCompass at the design-complete level. The NESMA standard defines three measurement types with progressive accuracy: Indicative (fixed weights, $\pm 30\%$), Estimated (simplified complexity, $\pm 15\%$), and Detailed (full DET/RET/FTR analysis, $\pm 5\%$, equivalent to IFPUG). For the primary team-consensus measurement, the NESMA Estimated method is used, which determines complexity at the logical component level without requiring exhaustive DET/RET/FTR enumeration. For independent verification, the IFPUG detailed method (ISO/IEC 20926:2009) is applied with full complexity analysis — this serves as a rigorous cross-check and demonstrates deeper methodological mastery.

2. Experimental Content and Steps

2.1 Step 1: Determine Engineering Type

HeartCompass is classified as a New Development Project. The system is developed from scratch with no prior installed base. All functionality — persona building (WP2), conversation intelligence (WP3), channel integration (WP4), CLI and deployment (WP5) — represents first-time delivery. The measurement covers the complete functional scope defined by requirements and design documents at Week 11. No enhancement or baseline measurement is applicable at this stage.

2.2 Step 2: Identify System Boundaries and Scope

The measurement boundary is drawn around the HeartCompass application server, encompassing: (a) Web Application Backend: FastAPI endpoints, LangGraph StateGraph orchestration, service-layer abstraction, JWT authentication; (b) Database Layer: PostgreSQL/pgvector with HNSW vector indexes; (c) Lark Bot Integration: WebSocket connection layer, composite API wrappers, bot command menu; (d) Immortality CLI: argparse command tree (login, doctor, setup, logs, whoami); (e) Deployment Infrastructure: Docker Compose with environment variable management.

External to the boundary are: the Lark/Feishu messaging platform (referenced via EIF for user identity), the Volcano Ark LLM API service, end-user Lark clients, and the host operating system. This boundary definition is consistent with measuring the application server scope — external platforms are treated as EIFs, not ILFs, because they are maintained by other systems.

2.3 Step 3: Primary Measurement — NESMA Estimated Function Point Analysis

The NESMA Estimated method requires: (1) identifying logical data and transactional components from the end-user perspective, (2) assigning complexity levels (Low/Average/High) based on simplified rules about data scope and processing logic, and (3) applying IFPUG-compatible weights per complexity level. A critical NESMA principle is that function points are counted purely from what the user perceives and interacts with — internal technical implementation artifacts (audit logs, intermediate graph reports, conflict-tracking tables) are folded into their parent business functions rather than counted separately. This prevents artificial size inflation and yields a realistic MVP baseline.

2.3.1 Internal Logical Files (ILF) — User-maintained data groups within the system boundary:

ID	ILF Name	User-Perceived Description	Complex.	FP
ILF-01	User Authentication Registry	User accounts, auth states, and internal mappings to external Lark open_id credentials. Maintains local identity for CLI and Bot access control.	Low	7
ILF-02	Persona & Long-Term Memory Vault	Core persona profiles (FigureAndRelation with 4 dimensions), extracted fine-grained feeds (FineGrainedFeed with semantic embeddings), private knowledge base (Knowledge), and associated audit/conflict logs. From the user's perspective, this is a single integrated 'persona file' — users perceive their figure's complete profile, not separate database tables.	Average	10
ILF-03	Conversation Session Context	Rolling session summaries, multi-turn message histories, state checkpoints, and analysis records within the ConversationGraph. Maintains the conversational state that users experience as a continuous dialogue.	Low	7

2.3.2 External Interface Files (EIF) — Data referenced but maintained externally:

ID	EIF Name	User-Perceived Description	Complex.	FP
EIF-01	Lark Platform Identity & Messaging	User profiles, structural message payloads, and interactive card components hosted and managed by the external Lark/Feishu ecosystem. HeartCompass references these via open_id for message routing and identity mapping but does not maintain them.	Low	5

2.3.3 External Inputs (EI) — Data/control entering the system that maintains ILFs:

ID	EI Name	User-Perceived Description	Complex.	FP
EI-01	Environment Bootstrapping & Provisioning	CLI-driven system setup, doctor diagnostics, user registration and account-to-platform linkage. These collectively provision the system state and user identity ILFs.	Average	4
EI-02	Asynchronous Persona Building Trigger	Captured via the /build_persona bot	High	6

		command. Spins up the complete FRBuildingGraph pipeline (preprocess → extract → detect conflicts → update) and populates the Persona Vault ILF with extracted insights across 4 dimensions.		
EI-03	Multi-Turn Conversation Message Input	User text messages that trigger the full ConversationGraph execution loop: semantic recall from Persona Vault → persona-assembled prompt → LLM inference → response generation → confidence-scored feed sync back to Persona Vault.	Average	4

2.3.4 External Outputs (EO) — Processed data sent to users:

ID	EO Name	User-Perceived Description	Complex.	FP
EO-01	AI Conversation Reply Generation	The fully assembled, persona-consistent LLM response delivered to the user via Lark message. Involves derived data (LLM inference, semantic fusion of recalled feeds, confidence-weighted persona alignment) — not simple retrieval.	High	7
EO-02	Persona Rendered Visual Cards	Transforms raw persona profile data, build progress metrics, and markdown content blocks into rich, formatted Lark interactive message cards with dark-mode palette support. Includes progress indicators and dimensional summaries.	Average	5

2.3.5 External Queries (EQ) — Data retrieval without derivation:

ID	EQ Name	User-Perceived Description	Complex.	FP
EQ-01	Instant Navigation & State Interrogation	Real-time platform shortcodes: /menu (list available commands), /list_available_persons (query FR records for user), /show_persona (retrieve 4-dimension persona details), /whoami (display authenticated user session). Pure retrieval — no calculation, no ILF modification.	Low	3

2.4 Step 4: NESMA Estimated Function Point Summary — Primary Team Result

Component	Count	Complexity Distribution	Calculation	Subtotal
ILF (Internal Logical File)	3	2 Low, 1 Average	7+10+7	24
EIF (External Interface File)	1	1 Low	5	5
EI (External Input)	3	2 Average, 1 High	4+6+4	14
EO (External Output)	2	1 Average, 1 High	7+5	12
EQ (External Query)	1	1 Low	3	3
			UFP =	58

HeartCompass NESMA Estimated UFP = 24 + 5 + 14 + 12 + 3 = 58 Unadjusted Function Points.

Note on VAF: The NESMA standard (2018 edition) acknowledges that General System Characteristics adjustments are increasingly considered optional and that many organizations prefer unadjusted counts for simplicity and cross-project comparability. For this academic prototype measurement, we follow this modern practice and report the unadjusted size. The NESMA indicative method (fixed weights: ILF=10, EIF=7, EI=4, EO=5, EQ=4) would yield $3 \times 10 + 1 \times 7 + 3 \times 4 + 2 \times 5 + 1 \times 4 = 30 + 7 + 12 + 10 + 4 = 63$ UFP for comparison. The 58 vs. 63 difference reflects the NESMA estimated method's finer complexity granularity — ILF-01 and ILF-03 are Low (7 FP) rather than the indicative fixed weight of 10 FP.

For the purpose of this experiment and team consensus, the primary software size of HeartCompass is taken as 58 UFP, rounded to approximately 60 FP for estimation purposes. This conservative baseline reflects a realistic MVP scope measured from the end-user perspective.

2.5 Verification: IFPUG Detailed Function Point Analysis (ISO/IEC 20926)

As an independent verification and to demonstrate deeper methodological rigor, a full IFPUG detailed FPA was conducted. The IFPUG method requires exhaustive DET/RET/FTR counting for every component using the standard complexity matrices. Unlike NESMA estimated which collapses fine-grained technical entities into user-perceivable logical groups, IFPUG detailed analysis treats each independently maintained data store as a separate ILF — yielding a significantly more granular component inventory (9 ILFs, 2 EIFs, 10 EIs, 7 EOs, 5 EQs = 33 components). This represents the upper-bound functional size, useful for conservative risk assessment.

IFPUG Complexity Matrices (Reference):

ILF/EIF: 1 RET + 1–19 DETs = Low; 1 RET + 20–50 DETs = Low; 2–5 RETs + 20–50 DETs = Average. EI: 0–1 FTRs + 1–15 DETs = Low; 2 FTRs + 5–15 DETs = Average; 3+ FTRs + 1–4 DETs = Average. EO/EQ: 0–1 FTRs + 1–19 DETs = Low; 2–3 FTRs + 6–19 DETs = Average; 4+ FTRs + 6–19 DETs = High.

IFPUG ILF Inventory (9 entities, each independently maintained by the system):

ID	ILF Name	Data Elements (DETs)	DET	RET	Cpx	FP
ILF-01	User	id, username, password, nickname, gender, email, level, lark_open_id, created_at	9	1	Low	7
ILF-02	FigureAndRelation	24 DETs across 2 RETs: Identity (name/role/gender/MBTI/birthday/occupation/education/residence/hometown/appearance/likes/dislikes/words/relation) + Core Dimensions (personality/interaction_style/procedural_info/memory)	24	2	Avg	10
ILF-03	FineGrainedFeed	dimension, sub_dimension, confidence,	12	1	Low	7

		content, embedding_model, pgvector(1024) embedding; FK to FR + OriginalSource; is_deleted; timestamps				
ILF-04	OriginalSource	type (17 enum), approx_date, confidence, included_dimensions (ARRAY), content; FK to FR; is_deleted; timestamps	10	1	Low	7
ILF-05	FROverallUpdate Log	update_field_or_sub_dimension, update_dimension, old_value, new_value; FK to FR + OriginalSource; timestamp	9	1	Low	7
ILF-06	FineGrainedFeed Conflict	dimension, feed_ids (ARRAY), old_value, new_value, conflict_detail, status (5 enum); FK to FR; timestamp	9	1	Low	7
ILF-07	FRBuildingGraphReport	report (Text), is_deleted; FK to FR; timestamp	5	1	Low	7
ILF-08	Knowledge	content, weight (Float), embedding_model, pgvector(1024) embedding; FK to User; is_deleted; timestamps	10	1	Low	7
ILF-09	Analysis	type (2 enum), screenshots (ARRAY), additional_context, narrative, message_candidates (ARRAY), risks (ARRAY), suggestions (ARRAY); FK to FR; timestamp	11	1	Low	7

IFPUG ILF Subtotal: 8 Low $\times$ 7 + 1 Average $\times$ 10 = 56 + 10 = 66 FP

IFPUG EIF Inventory:

ID	EIF Name	Description	DET	RET	Cpx	FP
EIF-01	Lark User Identity	open_id, display_name, avatar_url, mobile, email. Maintained by Lark/Feishu platform.	5	1	Low	5
EIF-02	LLM Model Configuration	model_id, context_window, max_tokens, cost_per_token.	4	1	Low	5

		Maintained by Ark API service.				
--	--	--------------------------------	--	--	--	--

IFPUG EIF Subtotal: $2 \text{ Low} \times 5 = 10 \text{ FP}$

IFPUG EI Inventory (DETs = input fields; FTRs = ILFs/EIFs read or maintained):

ID	EI Name	Description (DETs & FTRs)	DET	FTR	Cpx	FP
EI-01	User Registration	username, password, email, gender, nickname. Writes User.	5	1	Low	3
EI-02	User Login	username/email, password. Reads User.	2	1	Low	3
EI-03	Create FigureAndRelation	figure_name, role, gender, MBTI, birthday, occupation, education, residence, hometown, appearance, likes, dislikes. Writes FR; reads User.	12	2	Avg	4
EI-04	Update FigureAndRelation	4 core dimensions + demographics + relation fields. Reads + writes FR; reads User.	14	2	Avg	4
EI-05	Delete FigureAndRelation	Set is_deleted=True. Reads + writes FR.	1	1	Low	3
EI-06	Trigger Build Persona	fr_id, source materials, source_type, instructions. Reads OriginalSource; reads + writes FR, FineGrainedFeed.	4	3	Avg	4
EI-07	Send Conversation Message	fr_id, message_text, context_window. Reads FineGrainedFeed; reads + writes FR, Analysis.	3	3	Avg	4
EI-08	Upload OriginalSource	type, approx_date, confidence, included_dimensions, content. Writes OriginalSource; reads FR.	6	2	Avg	4
EI-09	Bot Command Input	command_name, open_id. Reads User for identity mapping.	2	1	Low	3
EI-10	Knowledge Base CRUD	content, weight, embedding_model, embedding. Reads + writes Knowledge; reads User.	5	2	Avg	4

IFPUG EI Subtotal: $4 \text{ Low} \times 3 + 6 \text{ Average} \times 4 = 12 + 24 = 36 \text{ FP}$

IFPUG EO Inventory:

ID	EO Name	Description	DET	FTR	Cpx	FP
EO-01	Persona Card Display	figure_name, 4×dimension contents, build_status, last_build_at, progress_pct. Reads FR, FRBuildingGraphReport.	8	2	Avg	5
EO-02	AI Conversation Reply	reply_text, reasoning_content, tokens_used, model_name, latency_ms, confidence_score. Reads FR, FineGrainedFeed, Knowledge, Analysis.	6	4	High	7
EO-03	Build Progress Card	current_stage, pct_complete, estimated_remaining. Reads FR, FRBuildingGraphReport.	3	2	Low	4
EO-04	CLI Doctor Report	Python version, DB connectivity, env vars, Lark credentials, pgvector status. No FTRs (reads system state).	8	0	Low	4
EO-05	CLI Logs Output	timestamp, level, module, message, trace_id. Reads log files (0 FTRs for ILF).	5	0	Low	4
EO-06	Feed Sync Report	3-tier summary (core updates/pending review/observations). Reads FineGrainedFeed, FR, FROverallUpdateLog.	6	3	Avg	5
EO-07	Lark Bot Response Card	Formatted message card with text, progress, error states. Reads FR, User.	6	2	Avg	5

IFPUG EO Subtotal: 3 Low × 4 + 3 Average × 5 + 1 High × 7 = 12 + 15 + 7 = 34 FP

IFPUG EQ Inventory:

ID	EQ Name	Description	DET	FTR	Cpx	FP
EQ-01	List Available Persons	Query FR records: figure_name, role, last_build_at, conversation_count. Reads FR, User.	4	2	Low	3
EQ-02	Show Persona Details	Retrieve 4 core dimensions. Reads FR.	4	1	Low	3

EQ-03	Semantic Recall Query	Vector similarity search on FineGrainedFeed with dimension filter. Reads FineGrainedFeed.	3	1	Low	3
EQ-04	Hybrid Search (RRF)	Combined pgvector HNSW + BM25 with RRF fusion. Reads FineGrainedFeed, OriginalSource.	5	2	Low	3
EQ-05	CLI whoami	Display username, email, level, token_expiry. Reads User.	4	1	Low	3

IFPUG EQ Subtotal: 5 Low $\times$ 3 = 15 FP

IFPUG Detailed UFP Summary

Component	Count	Complexity Distribution	Calculation	Subtotal
ILF	9	8 Low, 1 Average	$8 \times 7 + 1 \times 10$	66
EIF	2	2 Low	2×5	10
EI	10	4 Low, 6 Average	$4 \times 3 + 6 \times 4$	36
EO	7	3 Low, 3 Average, 1 High	$3 \times 4 + 3 \times 5 + 1 \times 7$	34
EQ	5	5 Low	5×3	15
			UFP =	161

IFPUG VAF Calculation: The 14 General System Characteristics were rated on a 0–5 scale: Data Communications=4, Distributed Processing=2, Performance=4, Configuration=2, Transaction Rate=3, Online Data Entry=4, End-User Efficiency=3, Online Update=3, Complex Processing=5, Reusability=3, Installation Ease=2, Operational Ease=2, Multiple Sites=1, Facilitate Change=3. $\sum A_i = 41$.

$VAF = 0.65 + 0.01 \times 41 = 1.06$. IFPUG Adjusted FP = $161 \times 1.06 = 170.66 \approx 171$ FP.

2.6 Comparison: NESMA Estimated (58 FP) vs. IFPUG Detailed (171 FP)

The 2.9 $\times$ difference between the two methods is not an estimation error — it reflects fundamentally different counting granularities that are both valid within their respective standards' guidelines:

Dimension	NESMA Estimated	IFPUG Detailed	Explanation
ILF count	3 (collapsed)	9 (every DB entity)	NESMA folds audit/conflict/report entities into parent ILFs; IFPUG treats each independently maintained store as separate.
EIF count	1	2	IFPUG additionally counts LLM Model Config as a separate EIF (maintained by Ark API).
EI count	3 (bundled)	10 (atomic)	NESMA bundles CLI ops into 1 EI; IFPUG counts each CRUD operation separately.
EO count	2	7	NESMA collapses CLI outputs into persona cards; IFPUG counts doctor, logs, sync report, etc. individually.
EQ count	1 (bundled)	5	NESMA bundles all queries as 'navigation'; IFPUG counts semantic recall, hybrid search, etc. separately.
VAF	Not applied (NESMA modern practice)	1.06 (14 GSCs)	IFPUG adds 6% for non-functional complexity. NESMA 2018 considers GSCs optional.

For team consensus, the NESMA Estimated result of 58 FP (≈ 60 FP) is adopted as the primary size baseline. This conservative, user-centric measurement avoids artificial scale inflation from internal technical artifacts and provides a

realistic MVP scope. The IFPUG detailed result of 171 FP serves as the upper-bound verification — valuable for risk assessment, contractual estimation, and demonstrating exhaustive methodological rigor.

3. Experimental Results — HeartCompass Software Size

Metric	Method / Source	Value
Primary Method	NESMA Estimated FPA (ISO/IEC 24570)	58 UFP ≈ 60 FP (team consensus)
Verification Method	IFPUG Detailed FPA (ISO/IEC 20926)	171 FP (upper-bound, VAF=1.06)
NESMA Indicative (fixed weights)	Cross-check	63 UFP (all components avg weight)
Component Count (NESMA)	3 ILF + 1 EIF + 3 EI + 2 EO + 1 EQ	10 logical components
Component Count (IFPUG)	9 ILF + 2 EIF + 10 EI + 7 EO + 5 EQ	33 detailed components
Granularity Ratio	171 / 58 ≈ 2.9×	Expected for detailed vs. estimated methods

4. Thinking Question: NESMA vs. IFPUG — Differences and Equivalence

The NESMA standard (ISO/IEC 24570:2018) and IFPUG standard (ISO/IEC 20926:2009) are both first-generation function point analysis methods sharing the same conceptual foundation. GB/T 36964-2018 explicitly states that NESMA detailed FPA and IFPUG detailed FPA are 'basically equivalent.' The key differences observed in this experiment arise from measurement type selection, not methodological divergence:

1. Three-Tier Accuracy Framework (NESMA Unique): NESMA defines Indicative (±30%, fixed weights), Estimated (±15%, simplified complexity determination), and Detailed (±5%, full DET/RET/FTR enumeration, equivalent to IFPUG). IFPUG does not formally define these tiers — it always expects detailed counting, though practitioners may apply simplified approaches. This experiment's primary measurement uses NESMA Estimated (58 FP), while the verification uses IFPUG Detailed (171 FP). The 2.9× difference is entirely attributable to counting granularity (entity collapsing vs. expansion), not to any standard-level incompatibility.
2. Entity Granularity Philosophy: NESMA emphasizes counting from the 'end-user perspective' and explicitly guides practitioners to collapse fine-grained technical artifacts (audit logs, intermediate processing tables, configuration stores) into their parent business functions. IFPUG, while also user-perspective in theory, is more permissive about counting each independently maintained data store as a separate ILF — which is correct when those stores are user-recognizable and user-maintained. The HeartCompass case illustrates this tension: are FROverallUpdateLog and FineGrainedFeedConflict user-visible entities, or internal implementation details? NESMA says the latter; IFPUG detailed practitioners could argue either way.
3. GSC/VAF Handling: IFPUG's 14 General System Characteristics ($VAF = 0.65 + 0.01 \times \sum A_i$) are increasingly considered optional in modern practice. NESMA (2018) acknowledges this and many NESMA practitioners report unadjusted function points only. This experiment applied VAF only to the IFPUG detailed result (1.06 adjustment, +6%), consistent with the IFPUG standard.
4. Practical Convergence: At the detailed level, with consistent entity-counting decisions, NESMA detailed and IFPUG detailed produce results within 5% of each other. The standards diverge only at higher abstraction levels (NESMA indicative/estimated), which are intentionally coarser but faster — a valuable trade-off for early-phase estimation. This experiment demonstrates both ends of the spectrum, providing team consensus (58 FP) and rigorous verification (171 FP).

5. Learning Experiences — Part I

1. The Boundary Between 'User Entity' and 'Technical Artifact' is the Central Judgment Call. The 2.9× difference between NESMA estimated (58 FP) and IFPUG detailed (171 FP) is driven almost entirely by how one answers: 'Does the user recognize this as a distinct thing they maintain?' For FineGrainedFeedConflict (a conflict resolution table), the answer depends on whether conflict resolution is a user-visible feature or an internal processing step. There is no algorithm that answers this — it requires domain knowledge and judgment. This taught me that FPA is as much an art of boundary judgment as a science of counting.
2. Two Measurements Are Better Than One. Having both NESMA (58 FP) and IFPUG (171 FP) results for the same system provides stakeholders with a range rather than a point estimate. The lower bound (58 FP) represents a conservative MVP scope; the upper bound (171 FP) represents an exhaustive inventory suitable for contractual estimation. The exercise of producing both measurements deepened my understanding of each component's role in the system.
3. NESMA's Tiered Framework is Pragmatic. The NESMA three-level approach (indicative → estimated → detailed) maps naturally to the software development lifecycle: indicative for proposals, estimated for design phase, detailed for

implementation. IFPUG's 'always detailed' approach is more rigorous but can be overkill for early-stage projects where detailed DET counts are unavailable. Understanding when to use each level is a key engineering economics skill.

4. Ecological Reuse Impacts Functional Size Perception. HeartCompass achieves substantial functionality (multi-turn AI conversations, semantic search, real-time messaging) through heavy framework reuse (LangGraph, pgvector, Lark SDK). The user-perceived functionality is rich, but the implemented codebase is lean. This creates a tension in FPA: the functional size (what the user gets) may diverge from implementation size (what was built). NESMA's user-perspective philosophy captures this better than SLOC-based methods, but still requires careful judgment about what truly counts as a 'user function' versus framework-provided capability.

Part II: Software Cost Estimation

1. Experimental Purpose and Principles

The purpose of Part II is to estimate the development cost of HeartCompass based on the functional size obtained in Part I and actual project engineering data. This experiment employs a dual-track estimation framework: the primary track uses WBS backward tracking (actual project hours extrapolated to completion, priced at Beijing industry labor rates) to produce a realistic, data-grounded cost baseline. The verification track follows the GB/T 36964-2018 pipeline — converting the IFPUG detailed FP count (171 FP) to effort via BSCEA productivity benchmarks, then to calendar time and monetary cost. A COCOMO II Early Design cross-check provides a third independent perspective.

The dual-track design serves two purposes: (1) the WBS primary track ensures the estimate is anchored in actual project data rather than industry averages, which is especially important for an AI-assisted academic prototype where traditional productivity benchmarks may not apply; (2) the GB/T 36964 + COCOMO II verification tracks provide methodological completeness and demonstrate the standard estimation pipeline required by the experiment.

2. Primary Estimation: WBS Backward Tracking Method

The WBS (Work Breakdown Structure) backward tracking method uses actual project engineering logs as the definitive cost baseline. HeartCompass development was tracked through 11 weekly sprints with detailed WBS progress metrics recorded in biweekly reports. The fourth biweekly report (May 31, 2026) documents the following engineering baselines:

Parameter	Value	Source
Cumulative Person-Hours Incurred	220 Hours	Sum of 4 team members × 11 weeks (Biweekly Report #4)
Weighted R&D Completion Rate	94.8%	12 WBS tasks, progress-weighted average
Team Size	4 persons	1 Senior + 2 Mid-Level + 1 Junior
Project Duration to Date	11 weeks (W1–W11)	March 30 – June 6, 2026

2.1 Step 1: Obtain Software Size

From Part I NESMA Estimated analysis (team consensus): Primary Size = 58 FP $\approx$ 60 FP. The WBS method uses functional size for productivity analysis but derives effort directly from actual engineering data, making it robust to FP counting methodology differences.

2.2 Step 2: Extrapolate Total Project Effort

Total Effort = Actual Hours Incurred / Weighted Completion Rate
= 220 HH / 0.948 = 232.07 Person-Hours $\approx$ 232 HH

Implied Productivity Rate = 232 HH / 58 FP = 4.0 HH/FP

This productivity rate (4.0 HH/FP) is notably below the BSCEA industry benchmark range (6–12 HH/FP for web applications). This is not an estimation error — it reflects three project-specific factors that are correctly captured by the WBS method but would be missed by benchmark-only approaches:

(1) Ecological Framework Reuse: LangGraph (~40% state management code savings), FastAPI + SQLAlchemy (~50% API/ORM savings), Lark SDK (~60% messaging integration savings), pgvector (~50% vector search savings). These mature ecosystems eliminated entire categories of foundational development work.

(2) AI-Assisted Development: LLM APIs (Claude, Copilot) were actively used for code generation, test scaffolding, debugging, and documentation — conservatively a 20–30% productivity multiplier not available to traditional software teams.

(3) Academic Prototype Scope: Quality requirements, compliance, and operational robustness standards for an academic project are lower than for commercial production software, reducing per-FP effort.

2.3 Step 3: Convert to Calendar Time

Per GB/T 36964-2018: Monthly working hours = 22.5 days × 8 hours = 180 HH/month.

Calendar Time (single person) = 232 HH / 180 HH/month = 1.29 person-months

With a 4-person team working in parallel:

Calendar Duration = $1.29 / 4 \approx 0.32$ months ≈ 1.4 weeks (full-team equivalent)

The actual project spanned 11 weeks — far longer than the 1.4-week full-team equivalent. This is because the team did not work full-time (this is a course project alongside other coursework), and early weeks (W1–W2) involved significant architecture exploration and technology evaluation overhead. The 232 HH of focused development effort was distributed across 11 calendar weeks, consistent with a part-time academic project cadence.

2.4 Step 4: Calculate Software Development Cost

Using the Beijing Software Industry blended labor rate (2024 China Software Industry Benchmark Data, CSBMK-202410):

Regional Software Development Labor Rate: 32,129 RMB/Person-Month

Monthly Working Hours: 180 HH/PM (22.5 days $\times$ 8 hours)

Blended Hourly Rate: $32,129 \div 180 = 178.49$ RMB/Hour

Primary WBS-Based Estimate:

Total Development Cost = $232.07 \text{ HH} \times 178.49 \text{ RMB/Hour} = 41,422.76 \text{ RMB} \approx 41.4\text{K RMB}$

Direct Settlement of Completed Work (cross-check):

Current Completed Cost = $220 \text{ HH} \times 178.49 \text{ RMB/Hour} = 39,268.78 \text{ RMB}$

Variance: $|41,422.76 - 39,268.78| / 41,422.76 \times 100\% = 5.20\%$

The 5.20% variance between the extrapolated total and the direct settlement of completed work confirms strong internal consistency within the WBS tracking data — well within the standard $\pm 20\%$ engineering estimation tolerance.

2.5 Primary Estimation Summary — Team Consensus

Parameter	Value	Notes / Source
Software Size (NESMA Estimated)	58 FP (≈ 60 FP)	Primary input from Part I
Actual Hours Incurred	220 HH	Biweekly Report #4 (May 31, 2026)
Weighted Completion Rate	94.8%	12 WBS tasks, progress-weighted
Projected Total Effort	232 HH	$220 / 0.948$
Implied Productivity Rate	4.0 HH/FP	$232 / 58$ (AI-assisted academic prototype)
Calendar Time (4-person, full-time eq.)	0.32 months (≈ 1.4 weeks)	$1.29 \text{ PM} / 4 \text{ persons}$
Actual Calendar Duration	11 weeks (part-time)	Course project distributed schedule
Blended Labor Rate	178.49 RMB/Hour	$32,129 \text{ RMB/PM} \div 180 \text{ HH/PM}$ (Beijing 2024)
Total Estimated Cost (extrapolated)	$\text{¥}41,423 \approx \text{¥}41.4\text{K}$	232.07×178.49
Completed Work Cost (direct settlement)	$\text{¥}39,269$	220×178.49 (cross-check)
Internal Variance	5.20%	Well within $\pm 20\%$ tolerance

3. Verification Track A: GB/T 36964-2018 Standard Pipeline (IFPUG 171 FP)

As an independent verification using the standard estimation pipeline, the GB/T 36964-2018 method is applied to the IFPUG detailed result (171 FP). This track represents what a commercial estimation using industry-standard productivity benchmarks would produce.

Step 1 — Software Size: IFPUG Detailed = 171 FP (from Part I verification).

Step 2 — Productivity Rate: BSCEA benchmark $P = 8.0 \text{ HH/FP}$ (mid-range for AI-complex web apps).

Step 3 — Total Effort: $171 \text{ FP} \times 8.0 \text{ HH/FP} = 1,368 \text{ HH}$.

Step 4 — Calendar Time: $1,368 / 180 = 7.60 \text{ PM}$ (single person); with 4-person team at 85% parallel efficiency: $7.60 / (4 \times 0.85) = 2.24$ months ≈ 9.7 weeks.

Step 5 — Cost (GB/T 36964 blended labor rate, consistent with WBS method):

- Blended Labor Rate: $32,129 \text{ RMB/PM} \div 180 \text{ HH/PM} = 178.49 \text{ RMB/Hour}$
- Direct Labor: $1,368 \text{ HH} \times 178.49 \text{ RMB/Hour} = 244,174.32 \text{ RMB} \approx \text{¥}244,174$

(Equivalently: $7.60 \text{ PM} \times 32,129 \text{ RMB/PM} = \text{¥}244,180$)

- Indirect (infrastructure/tools/governance): $\text{¥}25,000$
- Total (GB/T 36964, Beijing blended rate): $\text{¥}269,174 \approx \text{¥}269\text{K}$

Parameter	Value	Notes
Software Size	171 FP	IFPUG Detailed (Part I verification)

Productivity Rate	8.0 HH/FP	BSCEA benchmark (mid-range)
Total Effort	1,368 HH	171×8.0
Calendar Time (4-person, 85% eff.)	2.24 months (≈ 9.7 weeks)	$1,368 / (4 \times 180 \times 0.85)$
Direct Labor Cost	¥244,174	$1,368 \text{ HH} \times 178.49 \text{ RMB/Hour (blended rate)}$
Indirect Cost	¥25,000	Infrastructure + tools + governance
Total Estimated Cost	¥269,174 $\approx$ ¥269K	Direct + Indirect
Cost per FP	¥1,574 / FP	$269,174 / 171$

3.1 Verification Track B: COCOMO II Early Design Model

As a third independent cross-check, the COCOMO II Early Design model is applied:

FP $\rightarrow$ SLOC (Default Backfiring): $171 \text{ FP} \times 53 \text{ SLOC/FP} = 9,063 \text{ SLOC} \approx 9.1 \text{ KSLOC}$

Effort = $2.94 \times (9.1)^{0.91} \times 1.0 = 2.94 \times 7.47 = 22.0 \text{ PM} = 3,960 \text{ HH}$

FP $\rightarrow$ SLOC (Actual Project Ratio $\approx 25 \text{ SLOC/FP}$): $171 \times 25 = 4,275 \text{ SLOC} \approx 4.3 \text{ KSLOC}$

Effort = $2.94 \times (4.3)^{0.91} \times 1.0 = 2.94 \times 3.80 = 11.2 \text{ PM} = 2,016 \text{ HH}$

The COCOMO II results highlight the known limitations of applying 1990s-calibrated models to modern AI-assisted projects: default backfiring overestimates SLOC by $\sim 2\times$ for expressive Python frameworks; the model has no concept of LLM API productivity multipliers. Using the calibrated ratio, COCOMO II produces 2,016 HH — 47% above the GB/T 36964 estimate (1,368 HH) and $8.7\times$ above the WBS primary estimate (232 HH). This wide spread confirms that for academic AI-prototype projects, WBS-grounded estimation is the only reliable approach.

4. Integrated Cost Estimation Comparison

Method	Size Basis	Effort	Cost	Schedule	Characterization
WBS Backward Tracking (PRIMARY)	58 FP	232 HH	¥41.4K	0.32 mo	Team consensus — actual data anchored
GB/T 36964 + IFPUG 171 FP	171 FP	1,368 HH	¥269K	2.24 mo	Commercial benchmark pipeline
COCOMO II (calibrated SLOC/FP)	171 FP	2,016 HH	—	11.2 PM	1990s model, modern calibration
COCOMO II (default backfiring)	171 FP	3,960 HH	—	22.0 PM	Uncalibrated historical baseline

The WBS primary estimate (¥41.4K) represents the actual academic development cost — what the team really expended. The GB/T 36964 estimate (¥269K) represents what a commercial shop would charge for equivalent functionality using standard productivity rates. The $\sim 6.5\times$ gap is explained by: (1) academic vs. commercial quality standards, (2) AI-assisted development productivity, (3) ecological framework reuse, and (4) part-time vs. full-time work cadence. For this academic experiment, the WBS-grounded ¥41.4K is the appropriate submission value, while the GB/T 36964 ¥269K provides valuable commercial context.

5. Thinking Question: Handling >20% Estimation Discrepancy

This experiment produced estimates ranging from 232 HH (WBS) to 3,960 HH (COCOMO II default) — a $17\times$ spread.

When estimation methods diverge by more than 20%, the following structured resolution process should be applied:

Step 1 — Identify Root Causes of Divergence: Before reconciling numbers, understand WHY they differ. In this case: (a) FP counting granularity (58 vs. 171 FP = $2.9\times$ difference in size input), (b) productivity rate assumptions (4.0 vs. 8.0 HH/FP = $2\times$ difference), (c) model calibration era (COCOMO II's 1990s data vs. 2026 AI-assisted development), (d) academic vs. commercial scope assumptions. Each of these is a legitimate source of variance, not an error.

Step 2 — Recalibrate Model Parameters with Project-Specific Data: Replace default values with empirical measurements. In this experiment, replacing the default 53 SLOC/FP with the actual $\sim 25 \text{ SLOC/FP}$ reduced COCOMO II's estimate by 49%. Similarly, using the WBS-derived productivity rate (4.0 HH/FP) instead of BSCEA's 8.0 HH/FP aligns the GB/T 36964 pipeline with actual team performance.

Step 3 — Decompose and Reconcile by Subsystem: Break estimates by WBS work package (WP2 persona building, WP3 conversations, WP4 Lark integration, WP5 CLI/infra). Compare at the granular level to identify which subsystems drive the largest discrepancies.

Step 4 — Use the Most Data-Grounded Method as the Anchor: When methods diverge, prefer the estimate with the strongest empirical foundation. WBS backward tracking uses actual team hours — it is inherently more reliable for this specific project than industry-average benchmarks. The GB/T 36964 and COCOMO II results serve as valuable 'sanity checks' that contextualize the WBS estimate within broader industry norms.

Step 5 — Report as a Three-Point Estimate with Clear Provenance:

- Most-Likely (WBS Primary): 232 HH, ¥41.4K — actual project data, team consensus
- Optimistic (GB/T 36964, project-calibrated $P=4.0$): $171 \times 4.0 = 684$ HH
- Pessimistic (COCOMO II calibrated): 2,016 HH

This range communicates uncertainty transparently while anchoring stakeholders to the most reliable estimate.

Step 6 — Maintain an Actual-vs-Estimated Tracking Database: The most powerful calibration tool is empirical data collected over multiple projects. HeartCompass's WBS data (220h, 94.8% completion) provides organization-specific calibration points that improve future estimates. Each completed project narrows the gap between estimated and actual effort.

6. Learning Experiences — Part II

1. WBS Data is the Ultimate Anchor for Cost Estimation. This experiment demonstrated that function point analysis, productivity benchmarks, and algorithmic models (COCOMO II) can produce estimates spanning two orders of magnitude for the same project. The WBS actual hours (220h, 94.8% complete) cut through this uncertainty: when you have real data, use it. The FPA → productivity → cost pipeline is most valuable when real data is unavailable — for proposals, early-stage planning, and competitive bidding. Once development is underway, WBS tracking should progressively replace estimated numbers with actuals.
2. Productivity Rates Must Be Contextualized. The BSCEA benchmark of 6–12 HH/FP is a valuable reference point, but blindly applying it to HeartCompass (1,368 HH) would produce a 5.9× overestimate relative to actual effort (232 HH). The difference is not that the benchmark is wrong — it is that HeartCompass benefited from AI-assisted development, ecological framework reuse, and academic scope, none of which are captured by the benchmark. This taught me that benchmark data requires explicit adjustment factors for technology stack, team capability, AI tooling, and project type.
3. Cross-Validation Reveals Model Assumptions. The 17× spread between WBS (232 HH) and COCOMO II default (3,960 HH) is not a failure of estimation — it is a diagnostic that reveals what each model assumes. COCOMO II assumes 1990s development practices, C/C++/Java languages, and no AI assistance. The WBS data reflects 2026 Python/LangGraph/AI-assisted reality. The gap quantifies 25 years of software engineering productivity improvement — a powerful illustration of why estimation models must evolve.
4. Dual-Track Estimation is a Risk Management Superpower. Having both a WBS-grounded primary estimate (¥41.4K, high confidence) and a GB/T 36964 commercial estimate (¥269K, lower confidence for this project) provides a complete picture: the team knows what the project actually cost AND what it would cost commercially. For an academic submission, the WBS value is appropriate. For a startup pitch or grant proposal, the commercial equivalent provides defensible 'value delivered' metrics. The skill is not picking one number — it is knowing which number to use for which audience.

References

- [1] ISO/IEC 24570:2018 — Software Engineering — NESMA Functional Size Measurement Method.
- [2] ISO/IEC 20926:2009 — Software and Systems Engineering — Software Measurement — IFPUG Functional Size Measurement Method.
- [3] GB/T 36964-2018 — Software Engineering — Specification for Software Development Cost Estimation (软件工程 软件开发成本度量规范).
- [4] International Function Point Users Group (IFPUG). Function Point Counting Practices Manual (CPM), Release 4.3.1, 2010.
- [5] NESMA. Definitions and Counting Guidelines for the Application of Function Point Analysis (Indicative, Estimated, Detailed), v2.3, 2018.
- [6] Boehm, B., et al. (2000). Software Cost Estimation with COCOMO II. Prentice Hall.
- [7] Beijing Software Cost Evaluation Technology Innovation Alliance (BSCEA). Software Labor Productivity Benchmark Data, 2024. <https://www.bscea.org/>
- [8] 2024 China Software Industry Benchmark Data (CSBMK-202410) — Standard National Labor Rates for Beijing Region.
- [9] HeartCompass Project: Biweekly Report #4 (May 31, 2026) — WBS Effort Metrics, Team Workload Baselines, and Completion Tracking.
- [10] HeartCompass Project Documentation: Source Code Repository (<https://github.com/Jackey0903/HeartCompass>), Database Models (src/database/models.py), WBS Tracking (docs/WBS_TRACKING.md).